Depo Robotu Simülatörü

Aysıla Kan
Bilgisayar Mühendisliği
Kocaeli Sağlık ve Teknoloji Üniversitesi
Kocaeli, Türkiye
250501016/ 250501016@kocaelisaglik.edu.tr

İshak Debreli
Bilgisayar Mühendisliği
Kocaeli Sağlık ve Teknoloji Üniversitesi
Kocaeli, Türkiye
240501018/ 240501018@kocaelisaglik.edu.tr

***Özet*—Bu çalışmada, bir depoda çalışan tek bir robotun kutuları alıp teslim noktalarına taşıdığı bir simülatör geliştirilmiştir. Program; depo haritasını, robot komutlarını ve kural parametrelerini dosyalardan okumakta, komutları batarya, çarpışma ve yük kurallarına göre adım adım çalıştırmakta ve sonuçları JSON ve CSV dosyalarına yazmaktadır. Uygulama Python 3'ün standart kütüphanesi ile geliştirilmiş; kullanıcı arayüzü için Tkinter, ekip çalışması için Git ve GitHub kullanılmıştır. Ayrıca bonus olarak, robotun Genişlik Öncelikli Arama (BFS) algoritmasıyla hedefe en kısa yoldan gitmesini sağlayan GIT x y komutu eklenmiştir. Yapılan testlerde örnek test paketindeki altı testin tamamı geçilmiş ve hatalı girdilerde programın çökmeden kullanıcıyı bilgilendirdiği görülmüştür.**

Anahtar Kelimeler—simülasyon, Python, Tkinter, dosya işlemleri, BFS

I. GİRİŞ

Günümüzde depolarda ürünlerin taşınması için otonom robotlar giderek daha fazla kullanılmaktadır. Bu robotlar sınırlı bir bataryayla çalıştıkları ve depo içinde duvar, raf gibi engellerle karşılaştıkları için hareketlerinin önceden planlanması ve test edilmesi önemlidir. Bu projede, iki boyutlu bir depo haritası üzerinde tek bir robotun hareketlerini canlandıran bir simülatör geliştirilmiştir. Robot kendisine verilen komutlarla ilerlemekte, dönmekte, kutu alıp bırakmakta ve şarj istasyonunda şarj olmaktadır.

Projenin amacı; iki boyutlu listeler, metin, JSON ve CSV dosyalarıyla çalışma, metin ayrıştırma, fonksiyonlarla programlama, hata yönetimi ve arayüz geliştirme konularını uygulamalı olarak öğrenmektir [5]. Bunun yanında proje iki kişilik bir ekip tarafından Git ve GitHub kullanılarak, düzenli commit'lerle geliştirilmiştir. Raporun devamında Bölüm II'de kullanılan yöntem, Bölüm III'te deneysel sonuçlar, Bölüm IV'te ise elde edilen sonuçlar sunulmuştur.

II. YÖNTEM

Program iki ana dosyadan oluşmaktadır. Robotun kurallarını ve tüm hesaplamaları içeren fonksiyonlar simulasyon.py dosyasında, kullanıcı arayüzü ise arayuz.py dosyasında yazılmıştır. Projede yalnızca Python 3'ün standart kütüphanesi kullanılmıştır [1].

A. Dosyaların Okunması

Harita dosyası harita_oku() fonksiyonu ile satır satır okunmakta ve her satır karakterlerine ayrılarak iki boyutlu bir listeye dönüştürülmektedir. Böylece haritadaki her hücreye izgara[y][x] şeklinde erişilebilmektedir. Okuma sırasında tüm satırların aynı uzunlukta olup olmadığı ve haritada tam olarak bir tane robot (R) bulunup bulunmadığı kontrol edilmekte, bu kurallara uymayan haritalarda ValueError hatası verilmektedir. Robotun başlangıç konumu bulunduktan sonra haritadaki R karakterinin yerine boş hücre yazılmaktadır. Kural parametreleri ayarlari_oku() fonksiyonunda json modülü ile okunmaktadır [3].

Komut dosyası komutlari_oku() fonksiyonunda her satır split() ile parçalara ayrılarak incelenmektedir. ILERI komutunun yanında tek bir pozitif tam sayı bulunması gerekirken SAG, SOL, AL, BIRAK ve SARJ komutları parametre almamaktadır. Bu kurallara uymayan satırlar atlanmakta ve satır numarası, satırın içeriği ve hatanın nedeni ile birlikte bir listeye kaydedilmektedir. Boş satırlar komut olarak değerlendirilmemekte ancak satır numaralarına dahil edilmektedir.

B. Robotun Durumu ve Kurallar

Robotun anlık bilgileri yeni_durum() fonksiyonunda oluşturulan bir sözlükte tutulmaktadır. Bu sözlükte robotun konumu, yönü, batarya seviyesi, yük taşıyıp taşımadığı, teslim ve çarpışma sayıları, toplam adım sayısı, simülasyonun durup durmadığı ve hücrelerin ziyaret sayıları yer almaktadır. Harita bu sözlüğe kopyalanarak eklendiği için simülasyon sırasında yapılan değişiklikler orijinal haritayı bozmamakta ve simülasyon istenildiğinde baştan başlatılabilmektedir. Yönler saat yönünde [K, D, G, B] listesinde tutulmuştur. Sağa dönüşte yönün listedeki indeksi bir artırılmakta, sola dönüşte bir azaltılmaktadır. Listenin sonundan başına geçebilmek için (indeks + 1) % 4 şeklinde mod 4 işlemi kullanılmıştır.

ILERI n komutu tek seferde değil, adım adım işlenmektedir. Her adımda robotun önündeki hücre kontrol edilmekte; bu hücre duvar ya da harita dışıysa robot olduğu yerde durmakta, komutun kalan adımları iptal edilmekte, çarpışma sayısı bir artırılmakta ve bataryadan çarpışma cezası düşülmektedir. Ceza bataryayı eksiye düşürecekse batarya sıfırlanmakta ve simülasyon durdurulmaktadır. Adım maliyeti, robot kutu taşıyorsa yük çarpanıyla çarpılmakta; maliyet bataryadan büyükse adım atılmadan simülasyon sonlandırılmaktadır.

AL komutu yalnızca kutu (K) üzerinde, SARJ komutu yalnızca şarj istasyonu (S) üzerinde çalışmaktadır. BIRAK komutu teslim noktasında (T) kutuyu teslim etmekte, boş bir hücrede ise kutuyu oraya bırakmaktadır. Bu komutlar yanlış yerde verildiğinde GECERSIZ olayı üretilmekte ve batarya harcanmamaktadır. Robotun girdiği her hücrenin ziyaret sayısı tutulmakta, en çok ziyaret edilen hücre belirlenirken eşitlik durumunda önce y, ardından x değeri küçük olan hücre seçilmektedir.

Kuralların birlikte nasıl işlediği Tablo I'deki örnekle gösterilmiştir. #R.K.T# haritasında batarya 20, adım maliyeti 1, yük çarpanı 2 ve çarpışma cezası 3 olarak alınmıştır. Robot iki adım ilerleyip kutunun üzerine geldikten sonra kutuyu almakta, ardından yüklü olduğu için adım başına 2 birim harcayarak iki adım daha ilerlemekte ve üçüncü adımda duvara çarparak 3 birim ceza almaktadır. Son olarak teslim noktasında kutuyu teslim etmektedir. Geliştirilen program bu örnekte de aynı sonuçları üretmiştir.

TABLO I
ÖRNEK KOMUT DİZİSİNİN ADIM ADIM İŞLENMESİ

Komut	Olay	Konum	Batarya
ILERI 2	HAREKET	(3, 1)	18
AL	AL	(3, 1)	18
ILERI 5	CARPISMA	(5, 1)	11
BIRAK	TESLIM	(5, 1)	11

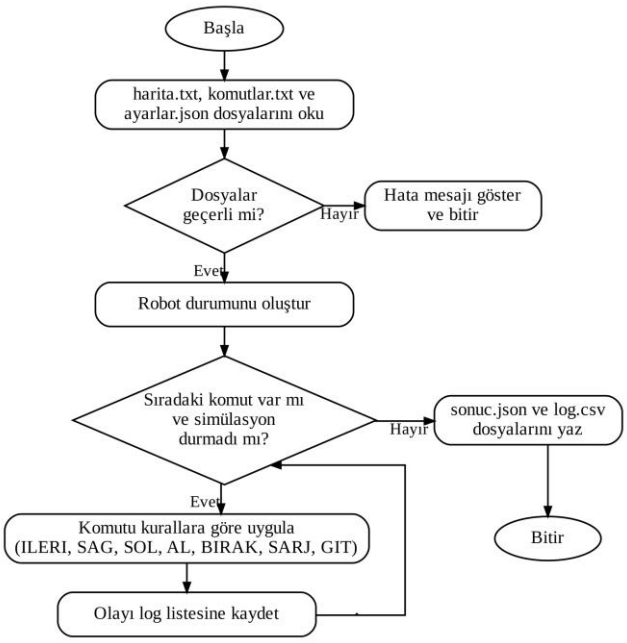
C. Simülasyon Akışı ve Çıktılar

komut_uygula() fonksiyonu gelen komutun türüne göre ilgili fonksiyonu çağırmakta ve gerçekleşen işlemi HAREKET, CARPISMA, DONUS, AL, TESLIM, SARJ veya GECERSIZ gibi bir olay adı olarak döndürmektedir. simulasyonu_calistir() fonksiyonu ise komutları sırayla uygulamakta ve her komuttan sonra robotun konumunu, yönünü, bataryasını ve olayı log listesine kaydetmektedir. Simülasyon durduğunda kalan komutlar çalıştırılmamaktadır.

Sonuçlar; konum, yön, batarya, yük, teslim, çarpışma, toplam adım, bataryanın bitip bitmediği, en çok ziyaret edilen hücre ve hatalı satırlar bilgileriyle sonuc.json dosyasına, her komutun kaydı ise adim, komut, x, y, yon, batarya ve olay sütunlarıyla log.csv dosyasına yazılmaktadır [3]. Programın akış diyagramı Şekil 1'de, veri_1 veri seti için üretilen log.csv dosyasının ilk satırları ise Tablo II'de gösterilmiştir.

TABLO II
VERİ_1 İÇİN ÜRETİLEN LOG.CSV DOSYASININ İLK SATIRLARI

adim	komut	x	y	yon	batarya	olay
1	ILERI 3	8	4	K	154	HAREKET
2	SOL	8	4	B	153	DONUS
3	ILERI 4	4	4	B	141	HAREKET
4	SAG	4	4	K	140	DONUS

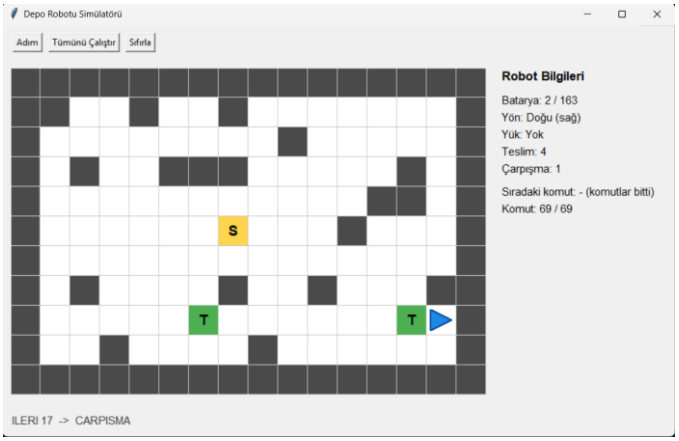


Şekil 1. Programın akış diyagramı

D. Kullanıcı Arayüzü

Kullanıcı arayüzü Tkinter kütüphanesi ile geliştirilmiştir [2]. Arayüz kendi içinde herhangi bir kural barındırmamakta, yalnızca simulasyon.py dosyasındaki fonksiyonları çağırmaktadır. Mantık ve arayüzün bu şekilde ayrılması sayesinde bonus olarak eklenen GIT komutu arayüzde hiçbir değişiklik yapılmadan çalışabilmiştir. Harita bir Canvas üzerinde her hücre 40 piksel olacak şekilde çizilmekte; duvarlar gri, kutular kahverengi, teslim noktaları yeşil ve şarj istasyonları sarı renkle gösterilmektedir. Robot ise baktığı yönü gösteren mavi bir üçgen olarak çizilmekte, kutu taşırken üzerinde küçük bir kutu görünmektedir.

Arayüzde Dosyaları Yükle, Adım, Tümünü Çalıştır ve Sıfırla butonları ile batarya, yön, yük, teslim, çarpışma ve sıradaki komutu gösteren bir bilgi paneli bulunmaktadır (Şekil 2). Dosya bulunamaması, bozuk harita veya hatalı komut satırı gibi durumlar try/except yapısıyla yakalanmakta ve program çökmeden kullanıcıya bir mesaj kutusu gösterilmektedir.

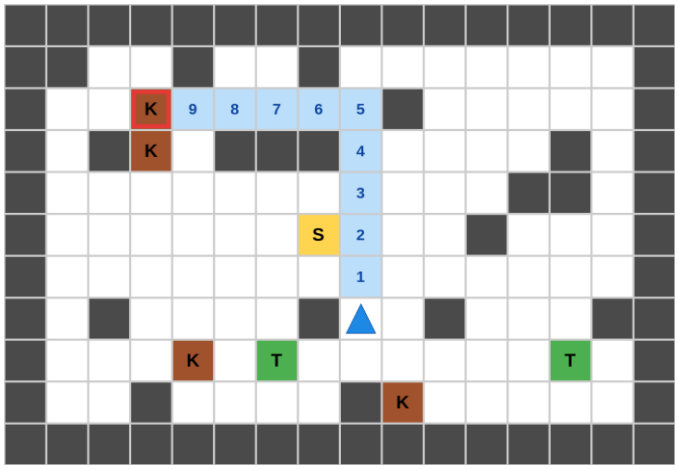


Şekil 2. Simülatörün kullanıcı arayüzü (veri_1, tüm komutlar çalıştırıldıktan sonra)

E. Bonus: En Kısa Yol (GIT x y)

Bonus olarak eklenen GIT x y komutu, robotu (x, y) hücresine duvarlardan kaçınarak en kısa yoldan götürmektedir. Yol, en_kisa_yol() fonksiyonunda Genişlik Öncelikli Arama (BFS) algoritmasıyla bulunmaktadır [4]. Algoritmada robotun bulunduğu hücreden başlanarak komşu hücreler bir kuyruk yardımıyla katman katman gezilmekte ve her hücreye hangi hücreden gelindiği bir sözlüğe kaydedilmektedir. Hedefe ulaşıldığında bu kayıtlar geriye doğru takip edilerek yol elde edilmektedir. BFS tüm yönlere eşit şekilde ilerlediği için hedefe ilk ulaşılan yol aynı zamanda en kısa yoldur.

Bulunan yol, git() fonksiyonunda mevcut dönme ve ilerleme fonksiyonları kullanılarak yürünmekte; böylece batarya ve yük kuralları bu komut için de aynen geçerli olmaktadır. Robot dönerken toplam dönüş sayısını en aza indirecek yön seçilmekte, hedefe ulaşılamıyorsa robot hareket etmemektedir. Şekil 4'te veri_1 haritasında GIT 3 2 komutu için bulunan yol gösterilmiştir. Robot (8, 7) hücresinden yukarı doğru beş adım ilerledikten sonra sola dönmekte ve aradaki duvarların çevresinden dolaşarak on adımda kutuya ulaşmaktadır.



Şekil 4. veri_1 haritasında GIT 3 2 komutu için BFS ile bulunan en kısa yol (mavi üçgen: robot, kırmızı çerçeve: hedef)

F. Sürüm Kontrolü ve Ekip Çalışması

Proje, GitHub üzerinde gizli (private) bir depoda geliştirilmiş ve ders asistanı depoya ortak çalışan olarak eklenmiştir. Kodun tamamı tek seferde yüklenmek yerine her fonksiyon tamamlandıkça ayrı bir commit atılmıştır. Böylece projenin gelişimi adım adım kayıt altına alınmış ve hangi bölümün hangi ekip üyesi tarafından yazıldığı commit geçmişinde görülebilir hale gelmiştir. İş bölümü yapılırken simülasyon, arayüz ve bonus bölümlerinin her ikisinde de iki üyenin de görev almasına dikkat edilmiş, böylece ekip üyelerinin projenin tüm kısımlarına hâkim olması amaçlanmıştır. Commit atmadan önce her değişiklik test_calistir.py ile denenerek daha önce çalışan bir özelliğin bozulmadığından emin olunmuştur. Ayrıca simülasyon çalıştırıldığında sonuç dosyalarının oluşup oluşmadığı ve hatalı girdilerde programın çökmediği de her aşamada kontrol edilmiştir.

III. DENEYSEL SONUÇLAR

Program, ders kapsamında verilen test_calistir.py betiği ve altı örnek testten oluşan paket ile denenmiştir. Testlerin tamamı başarıyla geçilmiştir (Tablo III, Şekil 3).

TABLO III
ÖRNEK TEST SONUÇLARI

Test	Açıklama	Sonuç
test01	Düz hareket	Geçti
test02	Duvara çarpma	Geçti
test03	Al, taşı, teslim et	Geçti
test04	Şarj istasyonu	Geçti
test05	Hatalı komut satırları	Geçti
test06	R olmayan harita	Geçti

```
PS C:\python> python test_calistir.py paket_ornek
Paket: paket_ornek   (6 test)

test01 düz hareket.......................... GEÇTİ
test02 duvara çarpma........................ GEÇTİ
test03 al-taşı-teslim et.................... GEÇTİ
test04 şarj istasyonu....................... GEÇTİ
test05 hatalı komut satırları............... GEÇTİ
test06 R olmayan harita..................... GEÇTİ

SONUÇ: 6/6  ->  35.00 / 35 puan
```

Şekil 3. Örnek test paketinin çalıştırılması sonucunda elde edilen çıktı

Program ayrıca verilen beş deneme veri seti ve GIT komutunu denemek için hazırlanan git_deneme veri seti ile çalıştırılmıştır. Elde edilen sonuçlar Tablo IV'te verilmiştir.

TABLO IV
DENEME VERİLERİYLE ELDE EDİLEN SONUÇLAR

Veri	Konum	Batarya	Teslim	Çarpışma	Adım	Hatalı satır
veri_1	(14,8)	2	4	1	63	3
veri_2	(11,1)	2	5	1	70	4
veri_3	(10,8)	94	4	1	67	3
veri_4	(10,1)	60	5	1	75	5
veri_5	(13,3)	86	3	1	34	5
git_deneme	(13,8)	111	4	0	64	0

Tüm veri setlerinde komut dosyalarındaki hatalı satırlar program çökmeden atlanmış ve satır numaralarıyla birlikte raporlanmıştır. Deneme veri setlerinin son komutu uzun bir ILERI komutu olduğundan robot her birinde duvara çarpmış ve çarpışma kuralının doğru çalıştığı görülmüştür. git_deneme veri setinde ise robot yalnızca GIT komutlarıyla yolunu kendisi bularak dört kutunun tamamını hiç çarpışma yaşamadan teslim etmiştir. Ayrıca robotu olmayan bir harita ve bulunmayan bir klasör seçildiğinde programın çökmeden uygun hata mesajını gösterdiği doğrulanmıştır.

IV. SONUÇ

Bu projede, bir depo robotunun hareketlerini dosyalardan okunan harita, komut ve ayarlara göre simüle eden, sonuçları dosyaya yazan ve arayüzde adım adım gösteren bir program

geliştirilmiştir. İstenen tüm özellikler ve bonus olarak en kısa yol bulma özelliği gerçekleştirilmiş, örnek testlerin tamamı geçilmiştir. Proje sürecinde iki boyutlu listeler, dosya işlemleri, hata yönetimi, Tkinter ile arayüz geliştirme ve BFS algoritması konularında deneyim kazanılmıştır. Git ile düzenli commit atarak çalışmak, iki kişinin aynı proje üzerinde düzenli bir şekilde ilerlemesini kolaylaştırmıştır.

Geliştirme sürecinde kodda Türkçe karakter kullanımı ve metinlerde fazladan bırakılan boşluklar gibi küçük hataların programın beklenmedik davranmasına yol açtığı görülmüş, bu hatalar her fonksiyonun ayrı ayrı test edilmesiyle bulunmuştur. İleride sisteme birden fazla robot eklenmesi, yolun batarya maliyetine göre seçilmesi ve robot hareketlerinin arayüzde animasyonla gösterilmesi gibi geliştirmeler yapılabilir.

KAYNAKÇA

[1] Python Software Foundation, "Python 3 Documentation," docs.python.org, 2026.

[2]Python Software Foundation, "tkinter — Python interface to Tcl/Tk," docs.python.org, 2026.

[3]Python Software Foundation, "json — JSON encoder and decoder; csv — CSV File Reading and Writing," docs.python.org, 2026.

[4]T. H. Cormen, C. E. Leiserson, R. L. Rivest ve C. Stein, *Introduction to Algorithms*, 4. baskı. Cambridge, MA, ABD: MIT Press, 2022.

[5]Kocaeli Sağlık ve Teknoloji Üniversitesi, "Programlama Lab-I Proje 1: Depo Robotu Simülatörü," ders dokümanı, 2026.